

Nome e cognome _____ Matricola _____

La durata della prova è di 2 ore. Scrivere in modo leggibile. E' obbligatorio restituire il testo e tutti i fogli forniti, anche se non si consegna il compito.

1. Considerati CPU-burst time (in ms) del set di processi descritto in tabella

Processo	Burst Time	Priorità	Tempo di arrivo
P1	6	2	0
P2	5	1	1
P3	3	3	3
P4	4	4	4
P5	2	1	5

- a) Disegnare 2 diagrammi di Gantt che illustrino l'esecuzione di questi processi usando gli algoritmi di priorità con prelazione (SRTF) e RR (quanto = 4 ms). Indicare l'istante di tempo di ciascuna commutazione fra un processo e l'altro
- b) Calcolare il tempo di attesa per ciascun processo e per ciascun algoritmo di scheduling.
- c) Calcolare il tempo di risposta per ciascun processo e per ciascun algoritmo di scheduling
- d) Discutere vantaggi e svantaggi dei due algoritmi

2. Tenuto conto che:

- il pid del processo padre è **9** e quello del primo figlio è **10**, quello del secondo figlio è **11**, quello del figlio del primo figlio è **12**.
- **Cosa stamperà il seguente programma? Giustificare in estrema sintesi la risposta**

```
int main() {
    int var1 = 2, var2 = 2;
    pid_t pid1, pid2;

    printf("Sono il processo padre.\n");

    pid1 = fork();

    if(pid1 > 0) {
        pid2 = fork();
        var1 = pid2 + pid1;
    }
    else {
        pid2 = fork();
        var2 = pid1 + pid2;
    }
    printf("pid = %d var1 = %d, var2 = %d \n", getpid(), var1, var2);
    return 0;
}
```

3. Dato il frammento di programma che segue:

- a) Discutere le sincronizzazioni presenti in questo programma
- b) Cosa stamperà il programma e perché?
- c) Cos'è una variabile di condizione e che ruolo ha nel programma?

```
#define NUM_THREADS 2
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int var = 4, turn = 1;

void *funcl(void* param){
    pthread_mutex_lock(&mutex);
```

```

    while (turn == 1) {
        pthread_cond_wait(&cond, &mutex);
    }
    var += turn;
    printf("func1: var = %d, turn = %d\n", var, turn);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

void *func2(void* param){
    pthread_mutex_lock(&mutex);
    turn = var;
    printf("func2: var = %d, turn = %d\n", var, turn);
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&mutex);
    return NULL;
}

int main(){
    pthread_t threads[NUM_THREADS];

    pthread_create(&threads[0], NULL, func1, NULL);
    pthread_create(&threads[1], NULL, func2, NULL);

    pthread_join(threads[0], NULL);
    pthread_join(threads[1], NULL);

    printf("Final values: var = %d, turn = %d\n", var, turn);
    return 0;
}

```

4. Domande a risposta multipla:

Selezionare le risposte corrette e giustificarle

- a. **Quale delle seguenti affermazioni descrive correttamente l'architettura a monolitica in un SO?**
 - 1) L'architettura monolitica separa i componenti principali del sistema operativo in moduli che comunicano tramite messaggi IPC (Inter-Process Communication).
 - 2) Assicura maggiore sicurezza, poiché ogni componente opera in isolamento rispetto agli altri.
 - 3) Tutti i componenti del SO sono integrati nello stesso spazio di memoria del kernel.
 - 4) Rende le operazioni più rapide ma meno sicure in caso di errori.
 - b. **Quale delle seguenti affermazioni è vera riguardo all'algoritmo di schedulazione EDF?**
 - 1) Assegna priorità più alte ai task che si ripetono meno frequentemente.
 - 2) Può garantire la schedulazione di task periodici se il carico di utilizzo del processore è inferiore al 100%.
 - 3) Assegna priorità ai task dinamicamente, privilegiando quelli con la scadenza più vicina.
 - 4) Non è adatto per sistemi real-time.
 - c. **Quale delle seguenti affermazioni è corretta riguardo l'architettura NUMA?**
 - 1) E' un modello in cui ogni processore ha una memoria fisica separata e non condivisa.
 - 2) I dati sono distribuiti in modo uniforme tra tutti i processori per migliorarne l'efficienza.
 - 3) I dati sono distribuiti in modo randomizzata per migliorarne l'efficienza.
 - 4) Per migliorare le prestazioni dei thread, si cerca di mantenere l'affinità tra i thread e i processori.
 - d. **Quale tra le seguenti affermazioni su spinlock e mutex è corretta?**
 - 1) Uno spinlock consente il blocco passivo del thread in attesa, mentre un mutex utilizza busy-waiting.
 - 2) Lo spinlock è più efficiente del mutex in sistemi monoprocesso perché riduce il tempo di attesa del thread.
 - 3) Lo spinlock utilizza busy-waiting per acquisire il lock quindi è meno efficiente di un mutex.
 - 4) Lo spinlock è vantaggioso rispetto al mutex per attese brevi in sistemi multicore perché evita il context switch.
5. **Discutere la differenza tra la comunicazione tra processi e thread evidenziando vantaggi e svantaggi.**
 6. **Spiegare la funzione dell'istruzione `compare_and_swap` illustrandone il funzionamento in pseudocodice. Illustrare un esempio pratico di impiego dell'istruzione.**